Name : Mokshitha kilaai
Reg : 192525413
Course : Computer Architecture

CO2 : Assessment Tool - 1

1. Signed 8-bit Arithmetic using 2's complement :-

Given,

A = +45

B = -23

Binary representation

45 = 00101101

-23 = 11101001

* Addition : (45) + (-23)

$$\begin{array}{r} 0010\ 1101 \\ 1110\ 1001 \\ \hline 10001\ 0110 \end{array}$$

Discard 1

Result = 0001 0110

= 22

∴ 45 + (-23) = 22

There is no overflow because the operands have different signs. Overflow in signed 2's complement addition occurs only when two number having the same sign produce a result with the oppos sign.

Scanned with OKEN Scanner

* Subtraction : (+45) - (-23)

2's Comp of -23 = 0001 0111

Now, 45 + 23

```
  0010  1101
  0001  0111
 ------------
  0100  0100
 ------------
```

Result = 01000100
= 68

The Processor Performs signed arithmetic using 2's complement representation and overflow is detected by checking the sign relationship between the operands and the results.

Q2. 4-bit Carry Look-Ahead Adder :-

A CLA is used to reduce the delay caused by carry propagation in a Ripple Carry Adder (RCA)

Concider,

A = 1011
B = 0110
$C_0 = 0$

Signals are

$P_i = A_i \oplus B_i$
$G_i = A_i \cdot B_i$

Carry equation:

$C_1 = G_0 + P_0 C_0$
$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$

Scanned with OKEN Scanner

$C_3 = G_2 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$

Final result

$$\begin{array}{r} 1011 \\ 0110 \\ \hline 10001 \end{array}$$

SUM = 0001

Carry = 1

* In Ripple Carry Adder, each carry waits for the previous carry, causing more delay. In a CLA, carries are calculated in advance using generate and propagate signals. Therefore CLA is faster and is suitable for high-performance processors although it requires more hardware.

23. Booth's Algorithms:

Multiplicand = -6 = 1010

Multiplier = +5 = 0101

Let

A → 0

B → 1010

Q → 0101

B = 1010

1'' = 0101

2'' = 0110

Operation	A	Q0	Q-1	SC
Initial	0 0 0 0	0 1 0 1	0	4
$Q_0Q_{-1}=10$	0 1 1 0	0 1 0 1	0	3
$A=A-B$	0 0 1 1	0 0 1 0	1	
A.S.R				
$Q_0Q_{-1}=01$	1 1 0 1	0 0 1 0	1	2
$A=A+M$				
A.S.R	0 1 0 0	1 0 0 1	0	
$Q_0Q_{-1}=01$	1 1 0 0	0 1 0 0	1	1
$A=A+M$				
A.S.R	1 1 1 0	0 0 1 0	0	
$Q_0Q_{-1}=00$	1 1 1 0	0 0 1 0	0	0

Result = 1110 0010

= -30

Final Answer: $(-6)\times(5)=-30$

Q4. Restoring and Non-Restoring Division :-

Given,

Dividend = 13

Divisor = 3

Binary:

13 = 1101

3 = 0011

A → 00000

B → 1101

Q → 00111

Scanned with OKEN Scanner

operation	A	Q	SC
Initial	00000	1101	4
L.S A = A - B	11110	1010	3
~~A is -ve~~ A = A + B L.S A = A - B	00001 00000	1010 0100	2
$Q_0 = A$ A = A - B	11101	1010	1
A = A - M	11110	0100	0
	00001	0100	

Quotient = 0100 = 4

Remainder = 00001 = 1

b)

A = 00000

Q = 1101

B = 00011

Operation	A	Q	SC
~~Initial~~	00000	1100	4
A = A - B ~~L.S~~ A = A + B	11110 00001	1010 1010	3
L.S A = A + M A = A - M	00000 11101	0101 1010	2
L.S A = A + M	11110	0100	1
	00001	0100	0

Scanned with OKEN Scanner

A = 00001

Therefore

Quotient = 0100 = 4

Remainder = 00001 = 1

* Restoring division restores the remainder whenever subtraction produces a negative result, while a non-restoring division avoids immediate restoration and performs addition/subtraction based on the sign of the remainder.

Q5. IEEE 754 Floating-Point Representation:

Given,

-13.25

Convert to Binary

$13.25 = 1101.01_2$

Normalize:

$-13.25 = -1.10101 \times 2^3$

Single Precision

Sign bit = 1

Exponent:

$3 + 127 = 130 = 10000010_2$

Scanned with OKEN Scanner

Double Precision:

Sign = 1

Exponent:

$3+1023 = 1026 = 10000000010_2$

* Floating Point Addition:

1. Compare exponents
2. Align Mantissas
3. Add / Subtract mantissas
4. Normalize the result
5. Round the result

Important issues are normalization, rounding.

Scanned with OKEN Scanner